

CAPSTONE PROJECT REPORT

Selenium Python Automation Framework with AdaptiveQA Self-Healing

Capstone Assignment 2: Selenium Python Framework Development

Unittest + PyTest + Page Object Model

1. Project Title

AdaptiveQA – Self-Healing Selenium Python Automation Framework for E-Commerce Testing

2. Introduction

Software testing plays an important role in ensuring the reliability and quality of web applications. Selenium provides an effective way to automate browser-based testing, while a well-structured automation framework improves maintainability, reusability, and scalability.

This project focuses on developing a **Selenium Python automation framework** for testing an e-commerce web application. The framework automates important functionalities such as **user login and product search** using the **Page Object Model (POM)** design pattern.

An additional feature called **AdaptiveQA – Self-Healing Locator Mechanism** was incorporated into the framework. It attempts alternative locators when the primary locator fails and records whether the locator was **STABLE, HEALED, or FAILED** in the HTML report.

3. Problem Statement

Traditional Selenium scripts can become difficult to maintain when the application's HTML structure or element attributes change. A change in a locator can cause automated test cases to fail even when the actual functionality of the application is working correctly.

Therefore, the objective of this project is to develop a reusable Selenium framework that:

- Automates web application functionality.
- Uses Page Object Model for maintainability.
- Separates test data and configuration from test scripts.
- Captures screenshots during failures.
- Generates HTML test reports.
- Provides logging and retry support.
- Includes a self-healing mechanism for locator failures.

4. Objectives

The main objectives of the project are:

1. To develop a Selenium automation framework using Python.
2. To implement the Page Object Model design pattern.
3. To automate login functionality.

4. To automate product search functionality.
5. To manage test data using CSV files.
6. To manage application and browser configuration using YAML.
7. To implement reusable utility classes.
8. To capture screenshots for failed tests.
9. To generate HTML execution reports.
10. To implement AdaptiveQA self-healing locator functionality.
11. To display locator health information in the test report.

5. Application Under Test

Application: TutorialsNinja Demo E-Commerce Application

URL:

<https://tutorialsninja.com/demo/>

The application was selected because it provides common e-commerce functionalities suitable for automation testing, including:

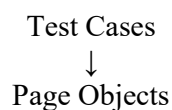
- User account/login
- Product search
- Product listings
- Shopping-related functionality

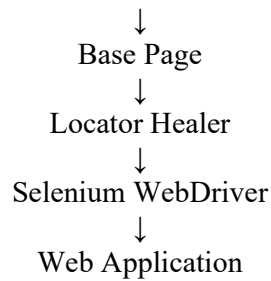
6. Technologies and Tools Used

Technology / Tool	Purpose
Python	Programming language
Selenium WebDriver	Browser automation
PyTest	Test execution
Unittest concepts	Test framework fundamentals
Page Object Model	Test structure and maintainability
YAML	Configuration management
CSV	Test data management
pytest-html	HTML reporting
Chrome WebDriver	Browser automation
Python Logging	Execution logging
GitHub	Source-code management
VS Code	Development environment

7. Framework Architecture

The framework follows a layered architecture:





Supporting components include:

Configuration → config.yaml

Test Data → login_data.csv

Utilities → CSV Reader / Screenshot

Framework → Driver Factory / Logger / Locator Healer

Reports → HTML Report

Evidence → Screenshots / Logs

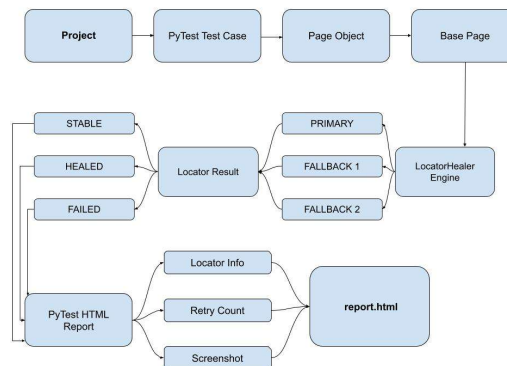


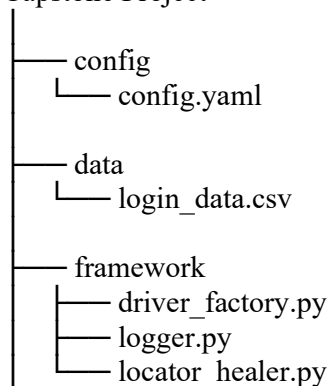
Fig. 1: Framework Architecture

The architecture separates test logic, page interaction, configuration, utilities, and reporting, making the framework easier to maintain and extend.

8. Project Structure

The final project is organized as follows:

Capstone Project



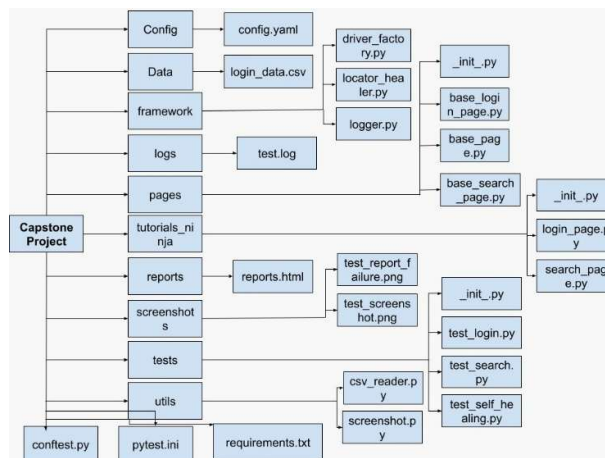
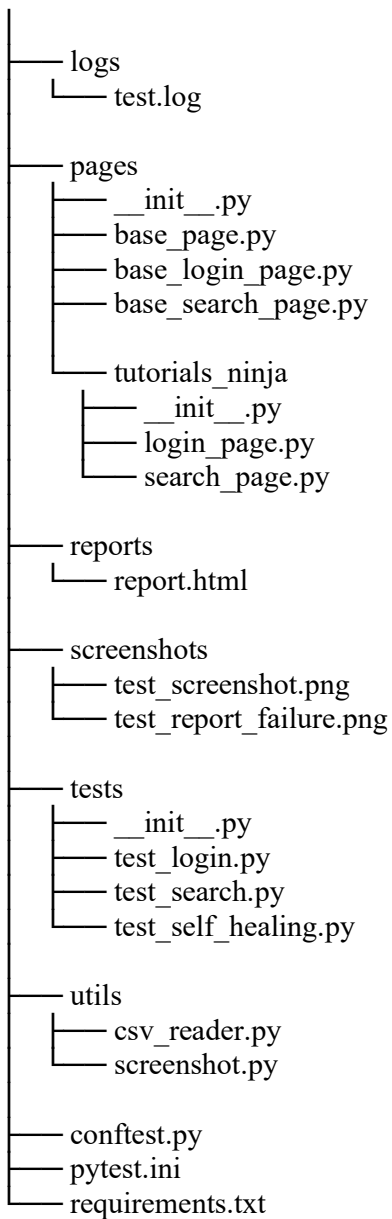


Fig.2: Project Structure

9. Implementation

9.1 Configuration Management

The framework uses a YAML configuration file to store application and browser settings.

The configuration contains information such as:

- Application URL
- Application name
- Browser name
- Headless execution option
- Implicit wait
- Page-load timeout

This prevents hard-coded configuration values from being repeated throughout the framework.

9.2 Driver Factory

The `driver_factory.py` module is responsible for creating and configuring the Selenium WebDriver.

The driver factory provides centralized browser management and applies the required browser settings before test execution.

This makes it easier to change browsers or browser configurations without modifying individual test cases.

9.3 Base Page

`base_page.py` contains common Selenium operations such as:

- Opening pages
- Clicking elements
- Entering text
- Waiting for elements
- Finding elements

The individual page classes inherit these reusable methods instead of implementing the same Selenium operations repeatedly.

10. Page Object Model

The project uses the **Page Object Model (POM)** design pattern.

Separate page classes were created for different application functionalities.

Login Page

The login page handles:

- Username input
- Password input
- Login action

Search Page

The search page handles:

- Search input
- Search button
- Product search functionality

The test cases therefore contain mainly business-level actions rather than low-level Selenium commands.

11. Test Data Management

Test data is maintained separately from the test scripts using:

data/login_data.csv

The CSV reader utility loads the required login information during test execution.

This provides a separation between:

Test Logic

and

Test Data

and allows additional test data to be added without changing the test implementation.

Sensitive credentials are not included in this report.

12. Test Scenarios

Test Case 1 – Login

Objective: Verify that a valid user can log into the application.

Steps

1. Launch the browser.
2. Open the application URL.
3. Navigate to the login page.
4. Read login data from the CSV file.
5. Enter username.

6. Enter password.
7. Click Login.
8. Validate successful execution.

Result: Passed.

Test Case 2 – Product Search

Objective: Verify that a user can search for a product.

Steps

1. Launch the application.
2. Locate the search field.
3. Enter the product name.
4. Click the search button.
5. Validate the search result.

Result: Passed.

Test Case 3 – Self-Healing Locator

Objective: Verify the AdaptiveQA locator recovery mechanism.

Steps

1. Execute a test using a primary locator.
2. Intentionally make the primary locator unavailable.
3. Allow the Locator Healer to attempt alternative locators.
4. Recover the element using a fallback locator.
5. Record the healing event.
6. Display the result in the HTML report.

Result: Passed with locator recovery.

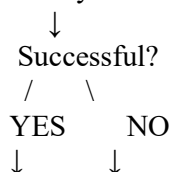
13. AdaptiveQA – Self-Healing Feature

The major additional feature of this project is **AdaptiveQA**, a self-healing locator mechanism.

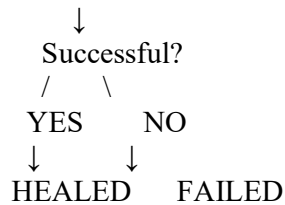
Normally, when a Selenium locator becomes invalid, the test fails immediately.

The AdaptiveQA mechanism instead follows:

Primary Locator



STABLE Fallback Locator



The framework records three possible locator states:

STABLE

The original locator successfully identifies the element.

HEALED

The original locator fails, but an alternative locator successfully identifies the element.

FAILED

Neither the original nor the fallback locators can identify the element.

14. Locator Health Reporting

The AdaptiveQA events are stored by the locator healer and integrated into the HTML report.

The report can display information such as:

Status Meaning

STABLE Primary locator worked

HEALED Primary locator failed and fallback locator worked

FAILED All locator attempts failed

Additional information includes:

- Original locator
- Recovered locator
- Retry/attempt count
- Test result
- Screenshot evidence

This makes the report useful not only for determining whether a test passed, but also for understanding the **health of the automation locators**.

Test Execution Summary table:

Test Case Functionality		Result AdaptiveQA
TC-01	Login	Passed STABLE

Test Case	Functionality	Result	AdaptiveQA
TC-02	Product Search	Passed	STABLE
TC-03	Self-Healing Locator	Passed	HEALED

The execution results demonstrate that the Login and Product Search tests completed successfully with stable locators, while the self-healing test successfully recovered from an invalid primary locator.

15. Screenshot on Failure

The framework captures screenshots when a test fails.

Screenshots are stored in:

screenshots/

Example evidence generated by the project includes:

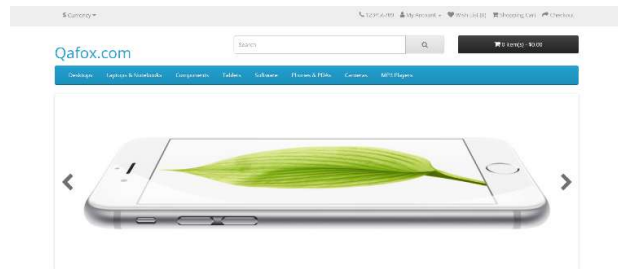


Fig.3: Screenshot captured as failure evidence during test execution.

Screenshots provide visual evidence of the browser state at the time of failure and make debugging easier.

16. Logging

A logging mechanism is implemented using: framework/logger.py

Execution information is stored in: logs/test.log

Logging helps in:

- Tracking test execution
- Identifying failures
- Debugging automation issues
- Understanding execution flow

17. HTML Reporting

The framework uses pytest-html to generate an HTML execution report. The report is generated under: reports/report.html

The report provides:

- Test execution status
- Passed tests
- Failed tests
- Execution duration
- Test details
- AdaptiveQA locator health
- Screenshot evidence where applicable

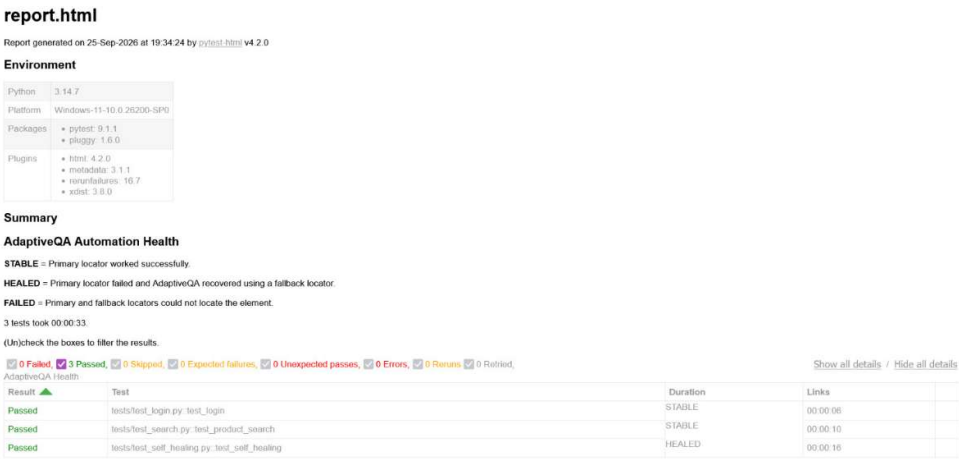


Fig.4: PyTest HTML execution report showing test results and AdaptiveQA locator health.

The generated report showed successful execution of the login and product-search tests and also recorded the self-healing test as **HEALED**.

18. Test Execution

The complete test suite can be executed using:

```
pytest -v -s
```

Individual tests can also be executed:

```
pytest tests/test_login.py -v -s
pytest tests/test_search.py -v -s
```

The complete execution performed during development successfully executed the implemented login and search functionality, along with the self-healing validation.

19. Test Execution Screenshots and Evidence

The following screenshots provide visual evidence of the Selenium automation execution performed on the TutorialsNinja application. They demonstrate successful test execution, login functionality, product search functionality, and the generated PyTest results.

```

testpaths: tests
plugins: html-4.2.0, metadata-3.1.1, rerunfailures-16.7, xdist-3.8.0
collected 3 items

tests/test_login.py::test_login PASSED
tests/test_search.py::test_product_search PASSED
tests/test_self_healing.py::test_self_healing PASSED

----- Generated html report: file:///C:/Users/JAITOSRI/OneDrive/Desktop/Desktop/Capstone%20Project/reports/report.html -----
===== 3 passed in 32.52s =====

```

Fig.5: PyTest terminal output showing successful execution of all three test cases.

The execution output shows that the Login, Product Search, and Self-Healing test cases passed successfully, with 3 tests completed in 32.52 seconds.

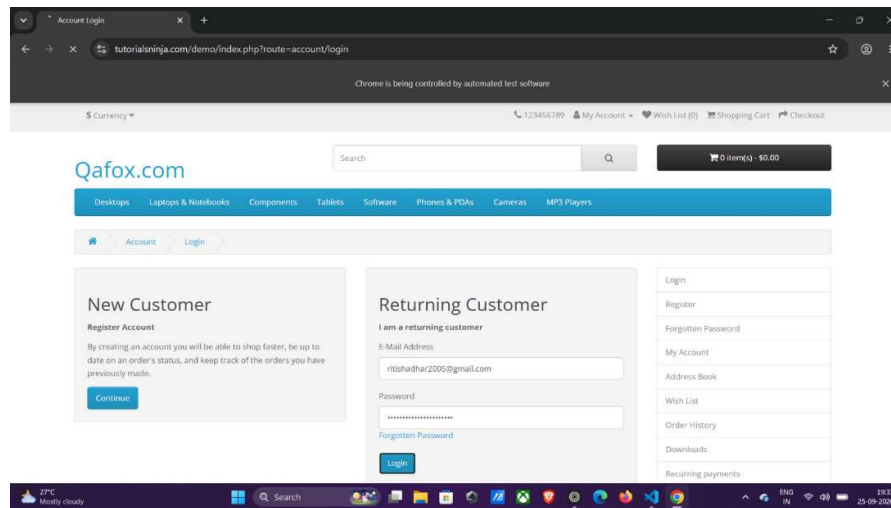


Fig.6: TutorialsNinja Login page during automated test execution.

The screenshot shows the login page with the test credentials populated automatically by the Selenium framework.

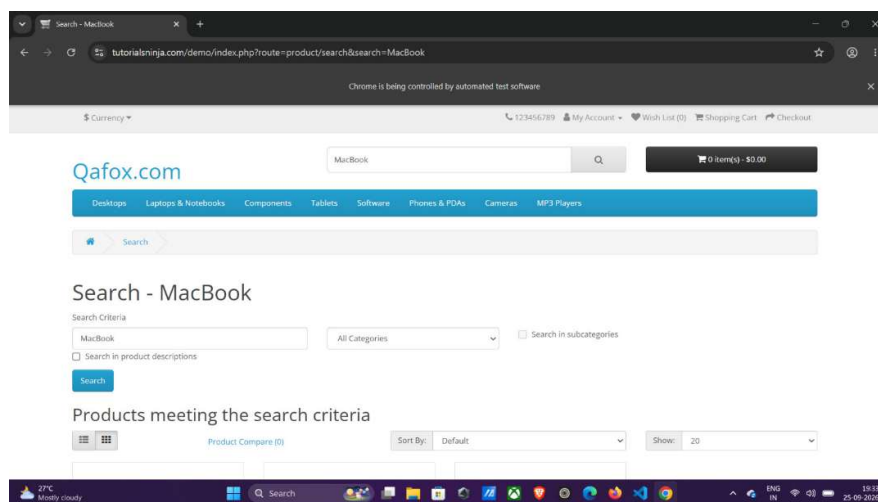


Fig.7: TutorialsNinja Product Search page after automated search execution.

The screenshot shows the search operation performed for "MacBook" and the resulting products displayed by the application.

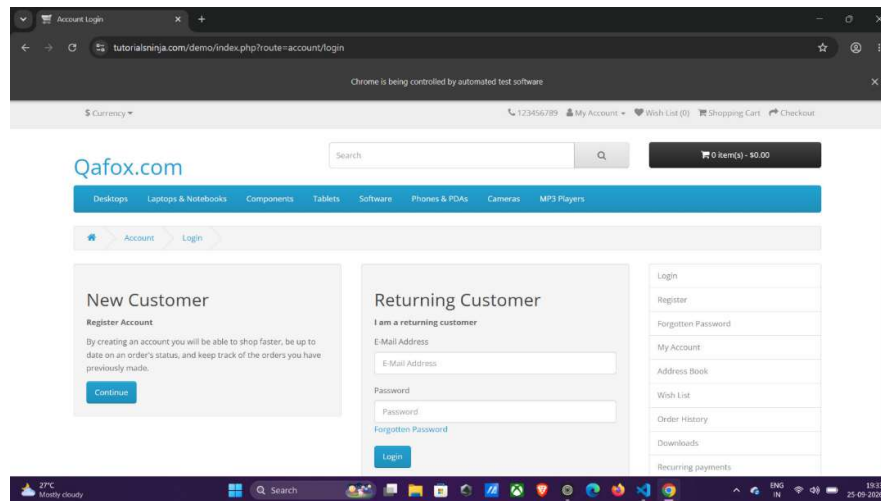


Fig.8: Tutorialsninja Login page before credential entry.

The screenshot shows the initial login interface before the automated test enters the username and password.

20. Results and Observations

The developed framework successfully demonstrated:

- Selenium browser automation
- Login automation
- Product search automation
- Page Object Model implementation
- YAML configuration management
- CSV test-data handling
- Reusable Selenium utilities
- Screenshot capture
- Execution logging
- HTML reporting
- Adaptive locator recovery
- Locator health reporting

The generated HTML report provides a centralized view of test execution and AdaptiveQA results.

21. Advantages of the Framework

The framework provides the following benefits:

- **Reusable:** Common Selenium operations are centralized.
- **Maintainable:** Page Object Model separates UI interaction from tests.
- **Scalable:** New pages and test cases can be added easily.

- **Data-driven:** Test data is maintained separately in CSV.
- **Configurable:** Browser and application settings are maintained in YAML.
- **Debuggable:** Logs and screenshots provide failure evidence.
- **Reportable:** HTML reports provide a readable execution summary.
- **Resilient:** AdaptiveQA attempts to recover from locator changes.

22. Limitations

The current implementation is focused primarily on:

- Login testing
- Product search testing
- Locator recovery
- Reporting and evidence generation

Additional application modules would be required for complete end-to-end e-commerce coverage.

23. Future Scope

The framework can be further enhanced by adding:

- Cross-browser testing
- Parallel execution
- API automation
- Database validation
- Visual regression testing
- CI/CD integration using GitHub Actions
- More advanced locator recovery
- Historical locator-health analytics
- Email or Slack notifications
- Additional e-commerce workflows such as cart and checkout

24. Conclusion

The project successfully developed a structured **Selenium Python automation framework** using PyTest, Page Object Model, configuration management, test-data management, screenshots, logging, and HTML reporting.

The addition of **AdaptiveQA Self-Healing Locator functionality** extends the framework beyond conventional Selenium automation. Instead of only reporting a locator failure, the framework attempts recovery using alternative locators and records the result as **STABLE**, **HEALED**, or **FAILED**.

Thus, the project demonstrates the practical application of Python, Selenium, PyTest, POM, automation framework design, test-data handling, reporting, and self-healing automation concepts in an e-commerce testing scenario.

25. References

1. Selenium Documentation — <https://www.selenium.dev/documentation/>
2. PyTest Documentation — <https://docs.pytest.org/>
3. Python Documentation — <https://docs.python.org/>
4. TutorialsNinja Demo Application — <https://tutorialsninja.com/demo/>
5. pytest-html Documentation — <https://pytest-html.readthedocs.io/>

26. Project Deliverables

The completed project contains the following major deliverables:

- Source code
- Page Object Model implementation
- Test cases
- CSV test data
- YAML configuration
- Selenium WebDriver setup
- Screenshot evidence
- Execution logs
- HTML test report
- AdaptiveQA self-healing implementation
- Locator health reporting
- Project architecture diagrams
- Test execution screenshots
- Demonstration videos

Final Outcome: The project provides a reusable Selenium Python automation framework with an additional **AdaptiveQA self-healing capability**, suitable for demonstrating practical automation-framework development skills.